

Spring Data JPA and Hibernate Assignment

Exercise 1: Overview and Setup

File name: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.18</version>
    <relativePath/>
  </parent>

  <groupId>com.ems</groupId>
  <artifactId>EmployeeManagementSystem</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <java.version>1.8</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
  </dependencies>
```

```

    <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
      </plugins>
    </build>
  </project>

```

File name: application.properties

```

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

```

File name: EmployeeManagementSystemApplication.java

```

package com.ems;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EmployeeManagementSystemApplication {

    public static void main(String[] args) {

        SpringApplication.run(EmployeeManagementSystemApplication.class,
            args);
    }
}

```

Exercise 2: Creating Entities

File name: Department.java

```

package com.ems.entity;

import com.fasterxml.jackson.annotation.JsonIgnore;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.ToString;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;

```

```

import javax.persistence.Id;
import javax.persistence.OneToMany;
import javax.persistence.Table;
import java.util.List;

@Entity
@Table(name = "department")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @JsonIgnore
    private List<Employee> employees;
}

```

File name: Employee.java

```

package com.ems.entity;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.Table;

@Entity
@Table(name = "employee")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;
}

```

```
    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

Exercise 3: Creating Repositories

File name: DepartmentRepository.java

```
package com.ems.repository;

import com.ems.entity.Department;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface DepartmentRepository extends
    JpaRepository<Department, Long> {

    Optional<Department> findByName(String name);
}
```

File name: EmployeeRepository.java

```
package com.ems.repository;

import com.ems.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface EmployeeRepository extends JpaRepository<Employee,
    Long> {

    List<Employee> findByDepartmentId(Long departmentId);
}
```

Exercise 4: Implementing CRUD Operations

File name: EmployeeController.java

```
package com.ems.controller;

import com.ems.entity.Employee;
import com.ems.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
```

```

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired
    private EmployeeRepository employeeRepository;

    @GetMapping
    public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(@PathVariable
Long id) {
        return employeeRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee) {
        return employeeRepository.save(employee);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Employee> updateEmployee(@PathVariable Long
id, @RequestBody Employee employeeDetails) {
        return employeeRepository.findById(id)
            .map(employee -> {
                employee.setName(employeeDetails.getName());
                employee.setEmail(employeeDetails.getEmail());

                employee.setDepartment(employeeDetails.getDepartment());
                return
                ResponseEntity.ok(employeeRepository.save(employee));
            })
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteEmployee(@PathVariable Long id)
{
        if (!employeeRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        employeeRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}

```

File name: DepartmentController.java

```
package com.ems.controller;
```

```

import com.ems.entity.Department;
import com.ems.repository.DepartmentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/departments")
public class DepartmentController {

    @Autowired
    private DepartmentRepository departmentRepository;

    @GetMapping
    public List<Department> getAllDepartments() {
        return departmentRepository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Department> getDepartmentById(@PathVariable
Long id) {
        return departmentRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Department createDepartment(@RequestBody Department
department) {
        return departmentRepository.save(department);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Department> updateDepartment(@PathVariable
Long id, @RequestBody Department departmentDetails) {
        return departmentRepository.findById(id)
            .map(department -> {
                department.setName(departmentDetails.getName());
                return
ResponseEntity.ok(departmentRepository.save(department));
            })
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteDepartment(@PathVariable Long
id) {
        if (!departmentRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        departmentRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}

```

```
}  
}
```

Exercise 5: Defining Query Methods

File name: Employee.java

```
package com.ems.entity;  
  
import lombok.AllArgsConstructor;  
import lombok.Data;  
import lombok.NoArgsConstructor;  
  
import javax.persistence.Entity;  
import javax.persistence.GeneratedValue;  
import javax.persistence.GenerationType;  
import javax.persistence.Id;  
import javax.persistence.JoinColumn;  
import javax.persistence.ManyToOne;  
import javax.persistence.NamedQueries;  
import javax.persistence.NamedQuery;  
import javax.persistence.Table;  
  
@Entity  
@Table(name = "employee")  
@NamedQueries({  
    @NamedQuery(name = "Employee.findByName", query = "SELECT e  
FROM Employee e WHERE e.name = :name")  
})  
@Data  
@NoArgsConstructor  
@AllArgsConstructor  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String name;  
  
    private String email;  
  
    @ManyToOne  
    @JoinColumn(name = "department_id")  
    private Department department;  
}
```

File name: EmployeeRepository.java

```
package com.ems.repository;  
  
import com.ems.entity.Employee;  
import org.springframework.data.jpa.repository.JpaRepository;
```

```

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;
import java.util.Optional;

public interface EmployeeRepository extends JpaRepository<Employee,
Long> {

    List<Employee> findByName(String name);

    List<Employee> findByDepartmentId(Long departmentId);

    @Query("SELECT e FROM Employee e WHERE e.email = :email")
    Optional<Employee> findByEmailCustom(@Param("email") String
email);
}

```

Exercise 6: Implementing Pagination and Sorting

File name: EmployeeRepository.java

```

package com.ems.repository;

import com.ems.entity.Employee;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;
import java.util.Optional;

public interface EmployeeRepository extends JpaRepository<Employee,
Long> {

    List<Employee> findByName(String name);

    List<Employee> findByDepartmentId(Long departmentId);

    @Query("SELECT e FROM Employee e WHERE e.email = :email")
    Optional<Employee> findByEmailCustom(@Param("email") String
email);

    Page<Employee> findByNameContainingIgnoreCase(String name,
Pageable pageable);
}

```

File name: EmployeeController.java

```

package com.ems.controller;

```



```

import com.ems.entity.Employee;
import com.ems.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired
    private EmployeeRepository employeeRepository;

    @GetMapping
    public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(@PathVariable
Long id) {
        return employeeRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee) {
        return employeeRepository.save(employee);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Employee> updateEmployee(@PathVariable Long
id, @RequestBody Employee employeeDetails) {
        return employeeRepository.findById(id)
            .map(employee -> {
                employee.setName(employeeDetails.getName());
                employee.setEmail(employeeDetails.getEmail());

                employee.setDepartment(employeeDetails.getDepartment());
                return
                ResponseEntity.ok(employeeRepository.save(employee));
            })
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteEmployee(@PathVariable Long id)
{

```

```

        if (!employeeRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        employeeRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/search")
    public Page<Employee> searchEmployees(
        @RequestParam String name,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size,
        @RequestParam(defaultValue = "name") String sortBy) {
        Pageable pageable = PageRequest.of(page, size,
Sort.by(sortBy));
        return
employeeRepository.findByNameContainingIgnoreCase(name, pageable);
    }
}

```

Exercise 7: Enabling Entity Auditing

File name: Employee.java

```

package com.ems.entity;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.EntityListeners;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.NamedQueries;
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import java.time.LocalDateTime;

@Entity
@Table(name = "employee")
@NamedQueries({
    @NamedQuery(name = "Employee.findByName", query = "SELECT e

```

```

FROM Employee e WHERE e.name = :name")
})
@EntityListeners(AuditingEntityListener.class)
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdDate;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;

    @CreatedBy
    @Column(updatable = false)
    private String createdBy;

    @LastModifiedBy
    private String lastModifiedBy;
}

```

File name: JpaAuditingConfig.java

```

package com.ems.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.domain.AuditorAware;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorProvider")
public class JpaAuditingConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        return new AuditorAwareImpl();
    }
}

```

File name: AuditorAwareImpl.java

```
package com.ems.config;

import org.springframework.data.domain.AuditorAware;
import java.util.Optional;

public class AuditorAwareImpl implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        return Optional.of("system");
    }
}
```

Exercise 8: Creating Projections

File name: EmployeeSummary.java

```
package com.ems.dto;

public interface EmployeeSummary {

    String getName();

    String getEmail();
}
```

File name: EmployeeDTO.java

```
package com.ems.dto;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class EmployeeDTO {

    private String name;

    private String email;
}
```

File name: EmployeeRepository.java

```
package com.ems.repository;

import com.ems.dto.EmployeeDTO;
import com.ems.dto.EmployeeSummary;
import com.ems.entity.Employee;
```

```

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;
import java.util.Optional;

public interface EmployeeRepository extends JpaRepository<Employee,
Long> {

    List<Employee> findByName(String name);

    List<Employee> findByDepartmentId(Long departmentId);

    @Query("SELECT e FROM Employee e WHERE e.email = :email")
    Optional<Employee> findByEmailCustom(@Param("email") String
email);

    Page<Employee> findByNameContainingIgnoreCase(String name,
Pageable pageable);

    List<EmployeeSummary> findEmployeeSummaryByDepartmentId(Long
departmentId);

    @Query("SELECT new com.ems.dto.EmployeeDTO(e.name, e.email) FROM
Employee e WHERE e.department.id = :deptId")
    List<EmployeeDTO> findEmployeeDTOByDepartmentId(@Param("deptId")
Long deptId);
}

```

File name: EmployeeController.java

```

package com.ems.controller;

import com.ems.dto.EmployeeDTO;
import com.ems.dto.EmployeeSummary;
import com.ems.entity.Employee;
import com.ems.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired

```

```

private EmployeeRepository employeeRepository;

@GetMapping
public List<Employee> getAllEmployees() {
    return employeeRepository.findAll();
}

@GetMapping("/{id}")
public ResponseEntity<Employee> getEmployeeById(@PathVariable
Long id) {
    return employeeRepository.findById(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

@PostMapping
public Employee createEmployee(@RequestBody Employee employee) {
    return employeeRepository.save(employee);
}

@PutMapping("/{id}")
public ResponseEntity<Employee> updateEmployee(@PathVariable Long
id, @RequestBody Employee employeeDetails) {
    return employeeRepository.findById(id)
        .map(employee -> {
            employee.setName(employeeDetails.getName());
            employee.setEmail(employeeDetails.getEmail());

            employee.setDepartment(employeeDetails.getDepartment());
            return
ResponseEntity.ok(employeeRepository.save(employee));
        })
        .orElse(ResponseEntity.notFound().build());
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteEmployee(@PathVariable Long id)
{
    if (!employeeRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    employeeRepository.deleteById(id);
    return ResponseEntity.noContent().build();
}

@GetMapping("/search")
public Page<Employee> searchEmployees(
    @RequestParam String name,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "5") int size,
    @RequestParam(defaultValue = "name") String sortBy) {
    Pageable pageable = PageRequest.of(page, size,
Sort.by(sortBy));
    return
employeeRepository.findByNameContainingIgnoreCase(name, pageable);
}

```

```

    }

    @GetMapping("/summary/{departmentId}")
    public List<EmployeeSummary> getEmployeeSummaries(@PathVariable
Long departmentId) {
        return
employeeRepository.findEmployeeSummaryByDepartmentId(departmentId);
    }

    @GetMapping("/dto/{departmentId}")
    public List<EmployeeDTO> getEmployeeDTOs(@PathVariable Long
departmentId) {
        return
employeeRepository.findEmployeeDTOByDepartmentId(departmentId);
    }
}

```

Exercise 9: Customizing Data Source Configuration

File name: SecondaryDataSourceConfig.java

```

package com.ems.config;

import
org.springframework.boot.autoconfigure.jdbc.DataSourceProperties;
import
org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import javax.sql.DataSource;

@Configuration
public class SecondaryDataSourceConfig {

    @Bean
    @ConfigurationProperties(prefix = "app.datasource.secondary")
    public DataSourceProperties secondaryDataSourceProperties() {
        return new DataSourceProperties();
    }

    @Bean
    public DataSource secondaryDataSource() {
        return
secondaryDataSourceProperties().initializeDataSourceBuilder().build();
    }
}

```

File name: application.properties

```

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver

```

```
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

```
app.datasource.secondary.url=jdbc:h2:mem:seconddb
app.datasource.secondary.driver-class-name=org.h2.Driver
app.datasource.secondary.username=sa
app.datasource.secondary.password=password
```

Exercise 10: Hibernate-Specific Features

File name: Employee.java

```
package com.ems.entity;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.DynamicInsert;
import org.hibernate.annotations.DynamicUpdate;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.EntityListeners;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.NamedQueries;
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import java.time.LocalDateTime;

@Entity
@Table(name = "employee")
@NamedQueries({
    @NamedQuery(name = "Employee.findByName", query = "SELECT e
FROM Employee e WHERE e.name = :name")
})
@EntityListeners(AuditingEntityListener.class)
@DynamicUpdate
@DynamicInsert
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Employee {
```



```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String name;

private String email;

@ManyToOne
@JoinColumn(name = "department_id")
private Department department;

@CreatedDate
@Column(updatable = false)
private LocalDateTime createdAt;

@LastModifiedDate
private LocalDateTime lastModifiedDate;

@CreatedBy
@Column(updatable = false)
private String createdBy;

@LastModifiedBy
private String lastModifiedBy;
}

```

File name: application.properties

```

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

app.datasource.secondary.url=jdbc:h2:mem:seconddb
app.datasource.secondary.driver-class-name=org.h2.Driver
app.datasource.secondary.username=sa
app.datasource.secondary.password=password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.jdbc.batch_size=20
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true

```